

### Question description

You are given a list consisting of  $n$  integers. Your task is to remove elements from the list at given positions, and report the removed elements.

### Constraints

**Question No:10**

- $1 \leq n \leq 2 \cdot 10^5$
- $1 \leq x_i \leq 10^9$
- $1 \leq p_i \leq n - i + 1$

### Input

The first input line has an integer  $n$ : the initial size of the list. During the process, the elements are numbered  $1, 2, \dots, k$  where  $k$  is the current size of the list.

The second line has  $n$  integers  $x_1, x_2, \dots, x_n$ : the contents of the list.

The last line has  $n$  integers  $p_1, p_2, \dots, p_n$ : the positions of the elements to be removed.

### Output

Print the elements in the order they are removed.

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

5  
2 6 1 4 2  
3 1 3 1 1

EXPECTED OUTPUT

1 2 2 6 4

Test Case 2

INPUT (STDIN)

6  
7 2 6 1 4 2  
11 3 1 3 1 1

EXPECTED OUTPUT

2 6 7 4 2 1

▼ Mandatory Test Cases

Test Case 1

KEYWORD

void build(int k,int  
1,int r)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

2

Test Case 2

TOKEN COUNT

330

Test Case 3

NLOC

37

## Code:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  #define MAXN 200005
5
6  int tree[4 * MAXN];
7
8  /* Build segment tree */
9
10 void build(int k, int l, int r) {
11
12     if (l == r) {
13
14         tree[k] = 1;
15
16         return;
17     }
18
19     int mid =
20         (l + r) / 2;
21
22     build(
23         2 * k,
24         l,
25         mid
26     );
27
28     build(
29         2 * k + 1,
```

```
30         mid + 1,
31         r
32     );
33
34     tree[k] =
35         tree[2 * k] +
36         tree[2 * k + 1];
37 }
38
39 /*
40  Find the x-th available element
41  and remove it.
42  */
43
44 int query(
45     int k,
46     int l,
47     int r,
48     int x
49 ) {
50
51     tree[k]--;
52
53     if (l == r)
54         return l;
```

```
56     int mid =  
57         (l + r) / 2;  
58  
59     int leftCount =  
60         tree[2 * k];  
61  
62     if (x <= leftCount) {  
63  
64         return query(  
65             2 * k,  
66             l,  
67             mid,  
68             x  
69         );  
70     }  
71     else {  
72  
73         return query(  
74             2 * k + 1,  
75             mid + 1,  
76             r,  
77             x - leftCount  
78         );  
79     }  
80 }
```

```
82  ∨ int main() {
83
84      int n;
85
86      scanf("%d", &n);
87
88      int *a =
89      |     malloc(n * sizeof(int));
90
91      /* Read array */
92
93      for (int i = 0; i < n; i++)
94      |     scanf("%d", &a[i]);
95
96      /* Initially every element is present */
97
98      build(
99      |     1,
100     |     0,
101     |     n - 1
102     );
103
104  ∨  /*
105     |     Read positions and remove them
106     */
```

```
108     for (int i = 0; i < n; i++) {
109
110         int position;
111
112         scanf("%d", &position);
113
114         int index =
115             query(
116                 1,
117                 0,
118                 n - 1,
119                 position
120             );
121
122         printf("%d", a[index]);
123
124         if (i != n - 1)
125             printf(" ");
126     }
127
128     printf("\n");
129
130     free(a);
132     return 0;
133 }
```

**Output:**

**Test Case-1:**

|  |           |  |
|--|-----------|--|
|  | 5         |  |
|  | 2 6 1 4 2 |  |
|  | 3 1 3 1 1 |  |
|  | 1 2 2 6 4 |  |

**Test Case-2:**

|  |              |  |
|--|--------------|--|
|  | 6            |  |
|  | 7 2 6 1 4 2  |  |
|  | 11 3 1 3 1 1 |  |
|  | 2 6 7 4 2 1  |  |

**Keshav Kumar Rauniyar**  
**CH.SC.U4CSE25236**